



MULUNGUSHI UNIVERSITY

SCHOOL OF ENGINEERING AND TECHNOLOGY

ICT 361: MOBILE APPLICATION PROGRAMING

LECTURER NAME: MR NYIRENDA

ARCHITECTURAL DESIGN REPORT

GROUP 1 : MEMBERS

GROUP 1

1	Mulemwa	Muchima	202402387	M	BSCS
2	Edwin	Kampamba	202406643	M	BSCS
3	Ivwananji	Chipasha	202400442	F	BSCS
4	Alinani	Sinyangwe	202305208	M	BSCS
5	Grace	Chilufya	202408399	F	BSCS
6	Charity	Namangolwa	202402615	F	BSCS
7	Memory	Mutemwa	202404528	F	BSCS
8	Mwila	Siaciti	202409048	F	BSCS
9	Hope	Nsana	202402859	F	BSCS
10	Judith	Mushabati	202403063	F	BSCS
11	Mcnaldo	Chilanga	202406887	M	BSCS
12	Collins	Kalela	202403266	M	BSCS
13	Nancy	Mailoni	202409163	F	BSCS
14	Onester	Kalunga	202408720	F	BSCS
15	Mayamiko	Phiri	202409779	F	BSCS
16	Atupele	Andende	202408882	M	BSCS

STUDEX APP MVVM ARCHITECTURE

ICT361 Mobile Application Development — Android Student Registration and Lab Group Management

1. Introduction

This report explains the Model-View-ViewModel (MVVM) architectural pattern used for the Android student registration and lab group management application, and walks through the layered architecture diagram and entityrelationship diagram (ERD) that support it. Together, these show how the app is structured internally, and how student and lecturer data is stored and kept consistent between the phone and the MySQL server.

2. What is MVVM?

MVVM (Model-View-ViewModel) is a design pattern that separates an app into three distinct responsibilities, so that the user interface, the app's logic, and its data never become tangled together. This separation makes the app easier to test, easier to maintain, and — critically for this assignment — able to preserve the user's work when the screen is recreated (for example, during a phone rotation).

2.1 Model

The Model represents the app's plain data — for example, a Student object holding a name, student number, programme and lab group. The Model contains no logic about how data is displayed or where it comes from; it is simply a container for information. In this project, the Model layer includes both the domain objects (e.g. Student) and the Room entities / Retrofit DTOs that represent that same data in local storage and over the network.

2.2 View

The View is what the user actually sees and interacts with — in Android terms, the Fragments and their XML layouts (forms, lists, buttons). The View's only responsibilities are to display whatever data it is given, and to forward user actions (a button tap, a form submission, a search query) upward to the ViewModel. The View contains no business logic and never talks to the database or the network directly.

2.3 ViewModel

The ViewModel sits between the View and the data layer. It holds the current state of the screen — for example, the selected lab group, a partially completed registration form, or the current search text — as LiveData objects that the View observes. Because the ViewModel is retained by the Android framework across configuration changes, it survives screen rotation, which directly satisfies the assignment's requirement to "preserve user work when Android recreates a screen." The ViewModel fetches or saves data by calling a Repository, and never talks to Room or Retrofit directly.

The overriding rule of MVVM in this project is a strict, one-directional chain of responsibility: the View only talks to the ViewModel, and the ViewModel only talks to the Repository. No layer is ever skipped.

3. Layered Application Architecture

The diagram below extends the MVVM pattern to show the full path data takes from the screen down to the remote database, including the offline-sync components required by the assignment.

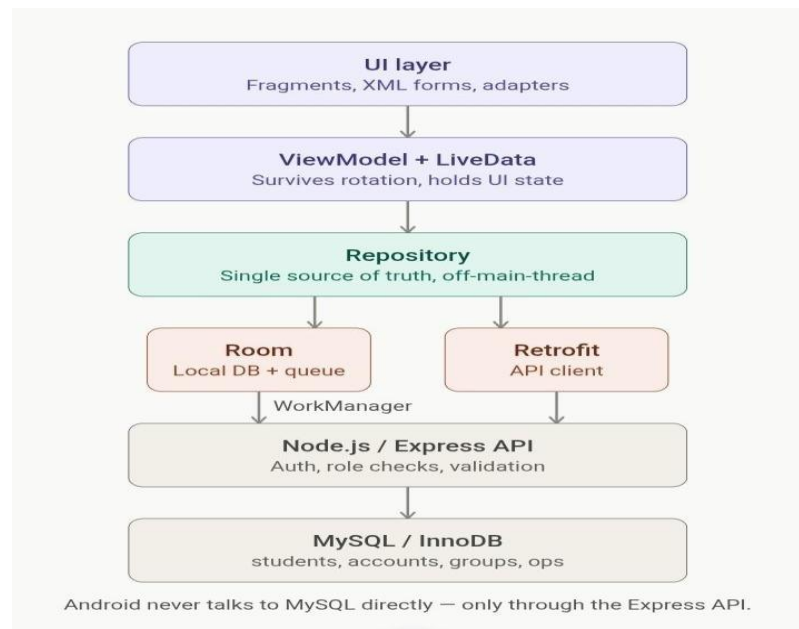


Figure 1 — Layered Android and backend architecture

3.1 UI layer

Fragments, XML layouts and RecyclerView adapters. This is the View in MVVM terms: it renders data and forwards user actions upward, with no logic of its own.

VISUAL SKETCH DESIGN

The visual sketch design gives a firsthand insight of the mobile App interface this was implemented using lucid chart tool. The App screens (Activity) are mainly divided 6 sections these are:

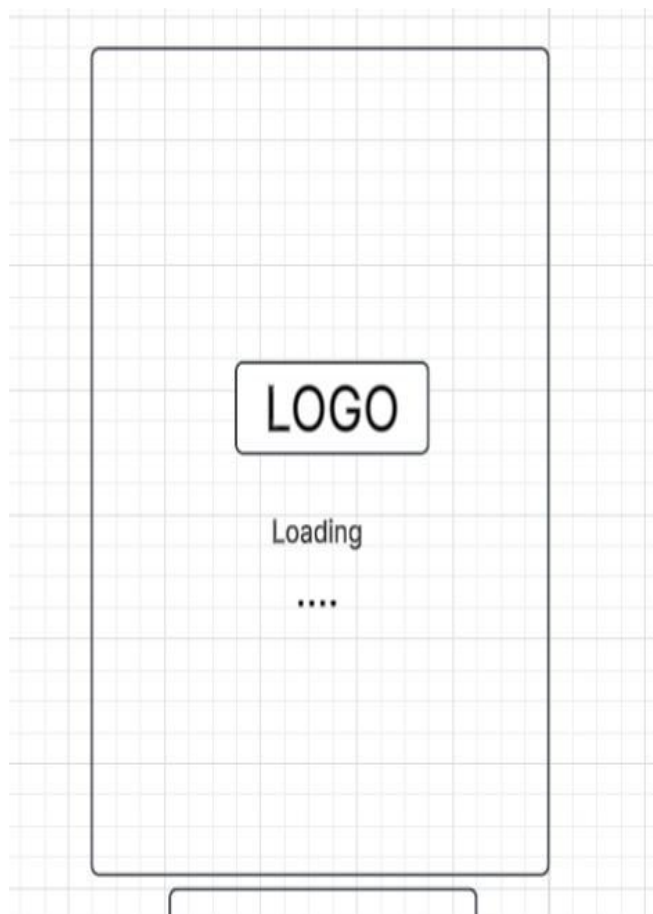
1. **Registration Screen**
2. **Sign-In Screen**
3. **Student Profile Screen**
4. **Lecturer Sign-In**
5. **Lecturer Roster Screen**
6. **Sync/Error Screen**

Visual representation



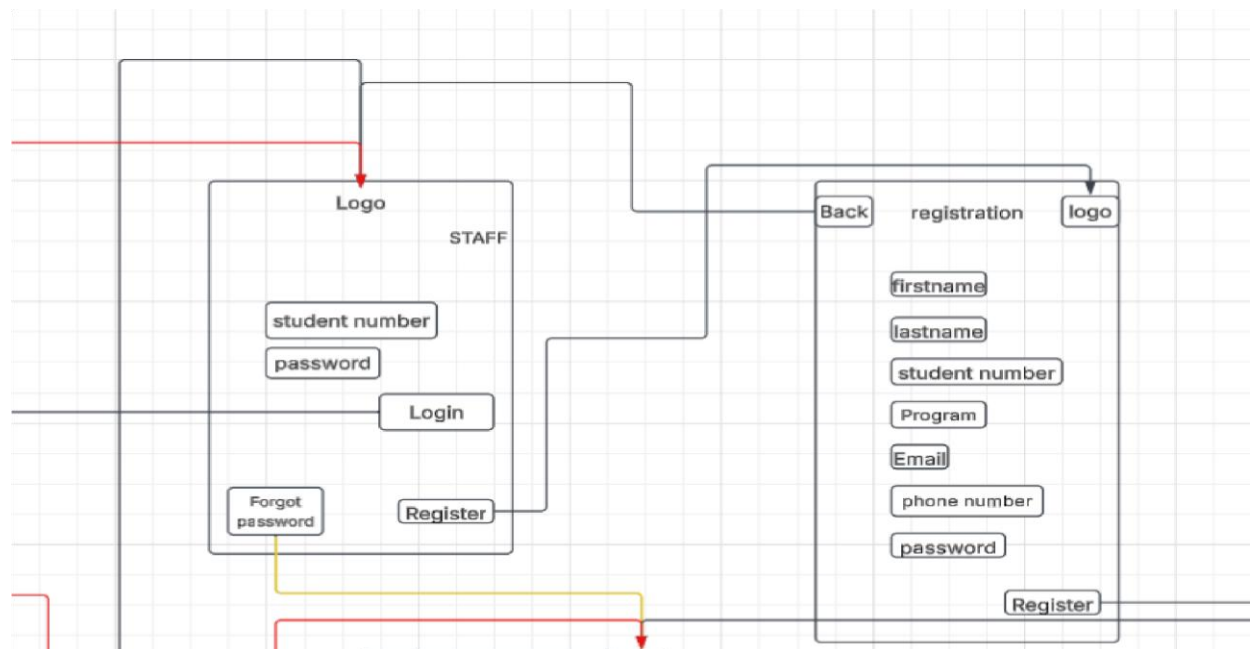
APP LOGO.....

THEME COLORS:BLUE,WHITE ,RED



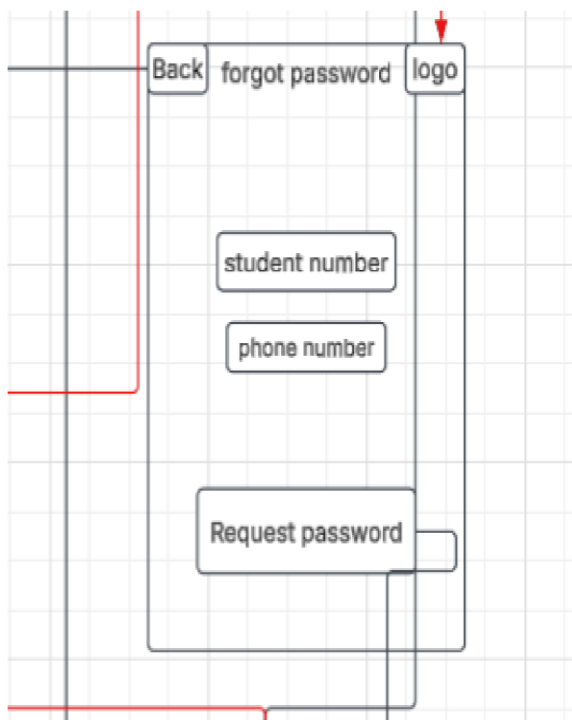
Activity 1 displaying the LOGO AND LOADING STATE

STUDENT REGISTRATION AND LOGIN

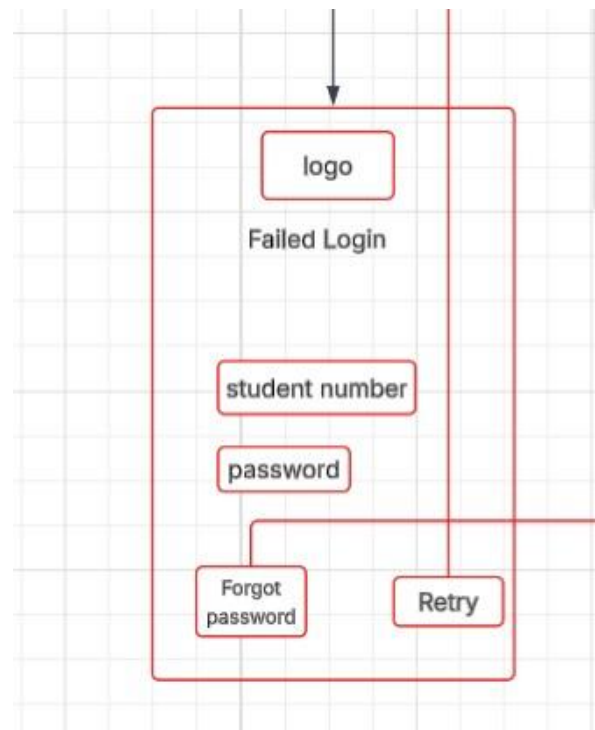


LOGIN SCREEN FIELDS: Student Number, Password, login button, forgot password button and register button.

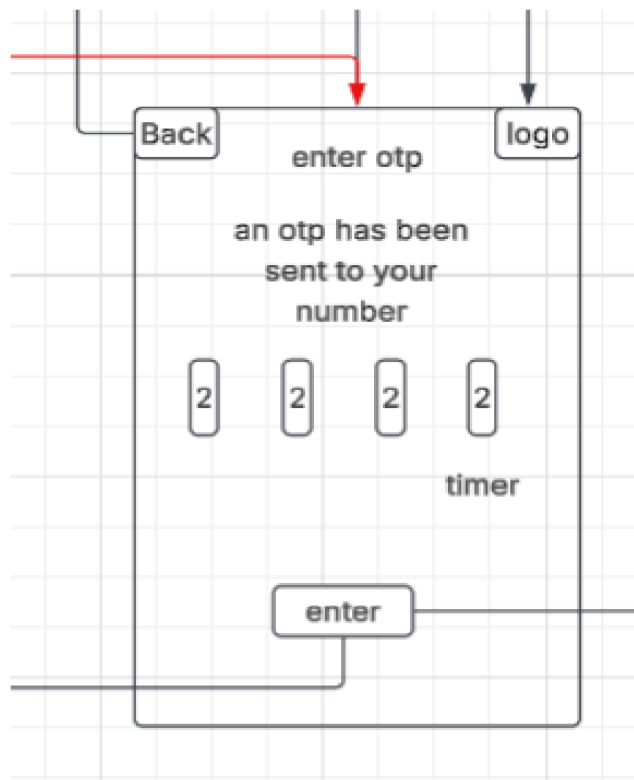
REGISTRATION SCREEN: First name, lastname , studentnumber , program,email ,phone number and password



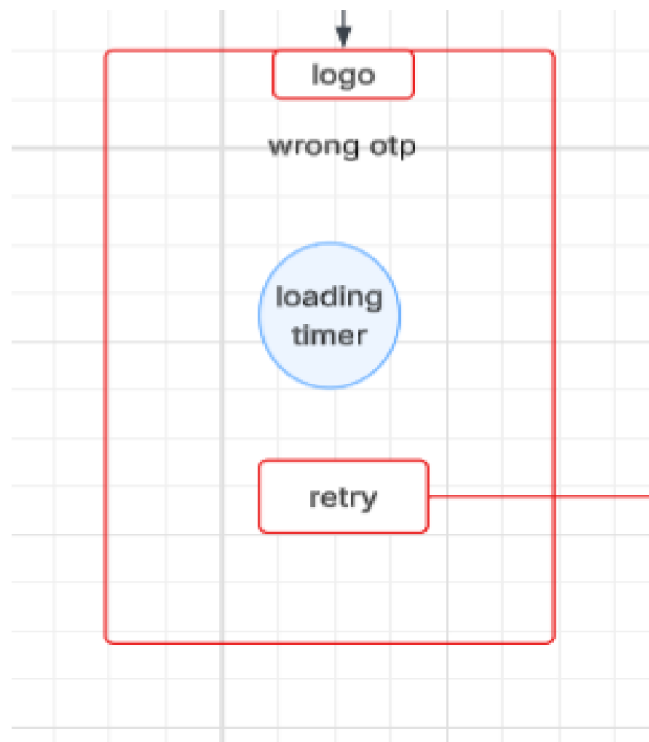
FORGOT PASSWORD SCREEN



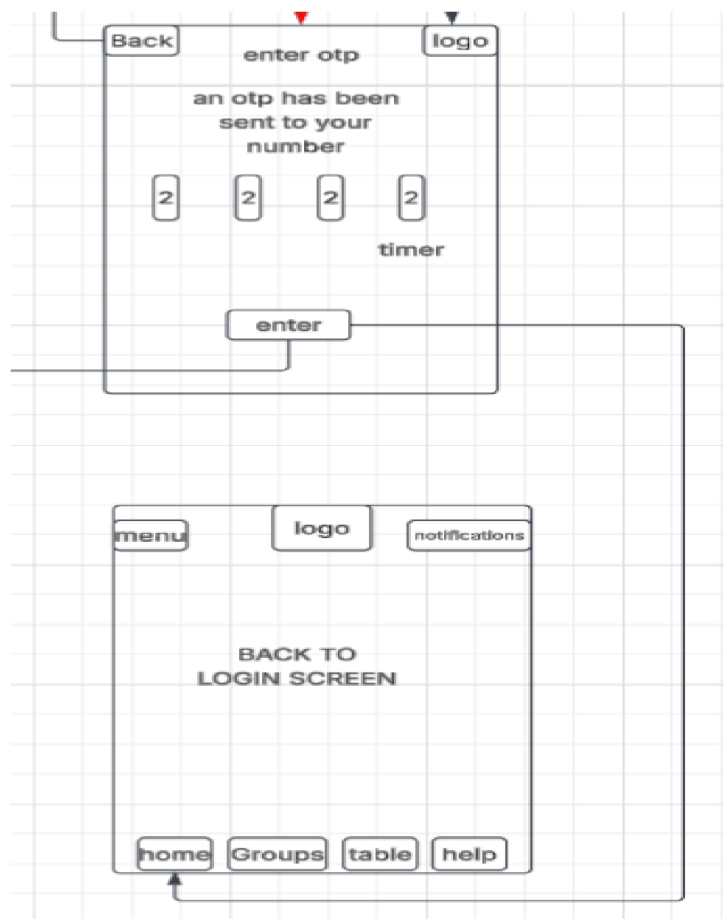
LOGIN ERROR



OTP REQUEST

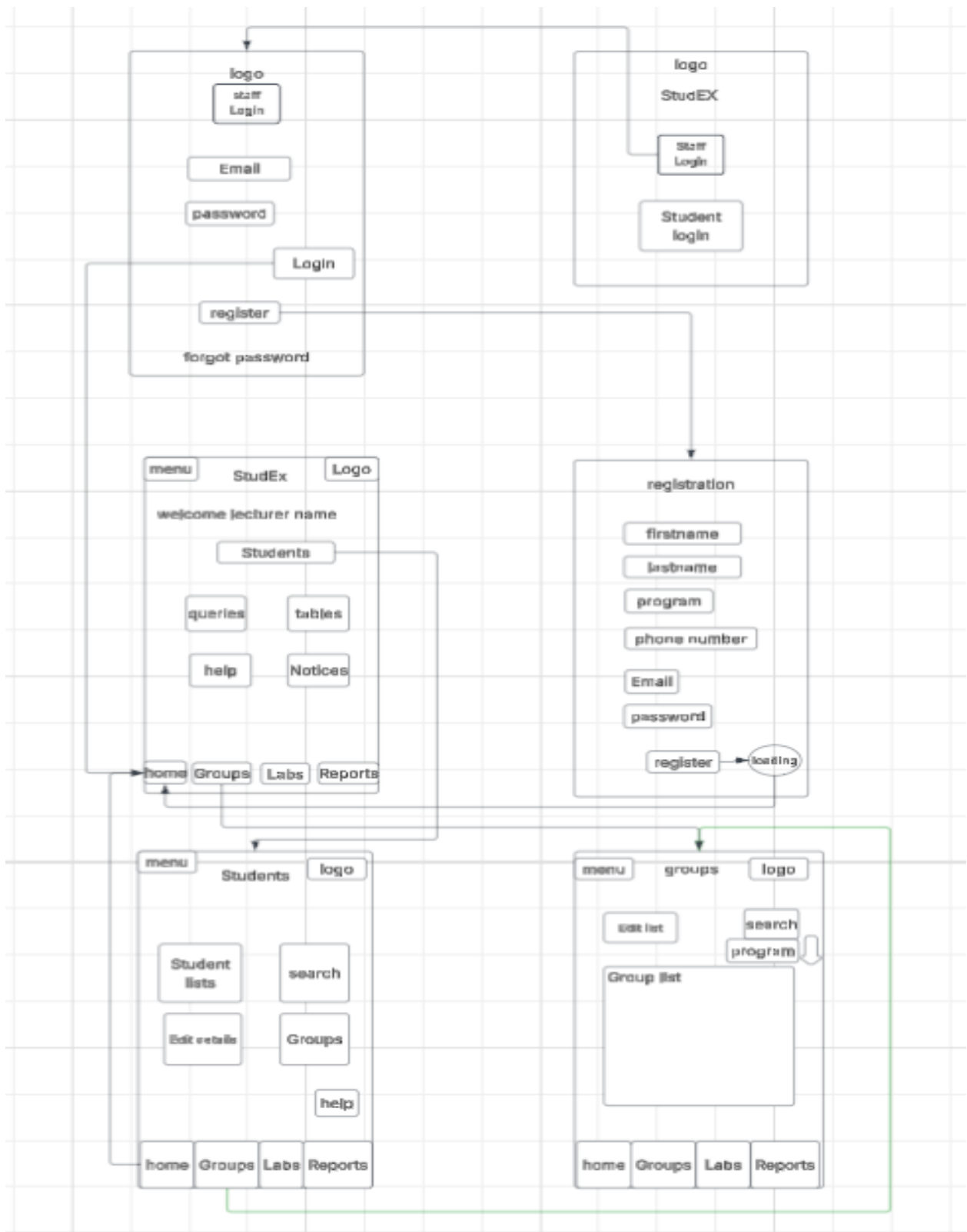


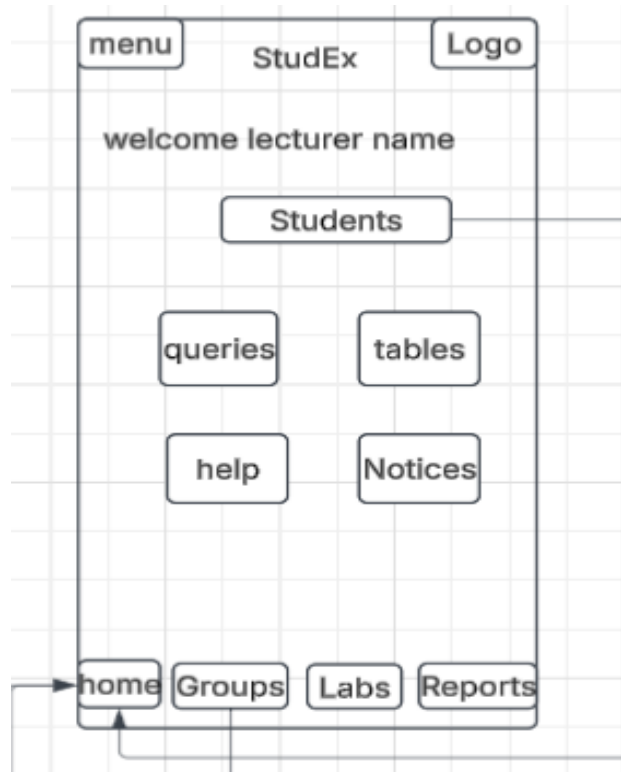
WRONG OTP ERROR



CORRECT OTP PASSWORD SENT TO SMS STATUS: RECOVERD

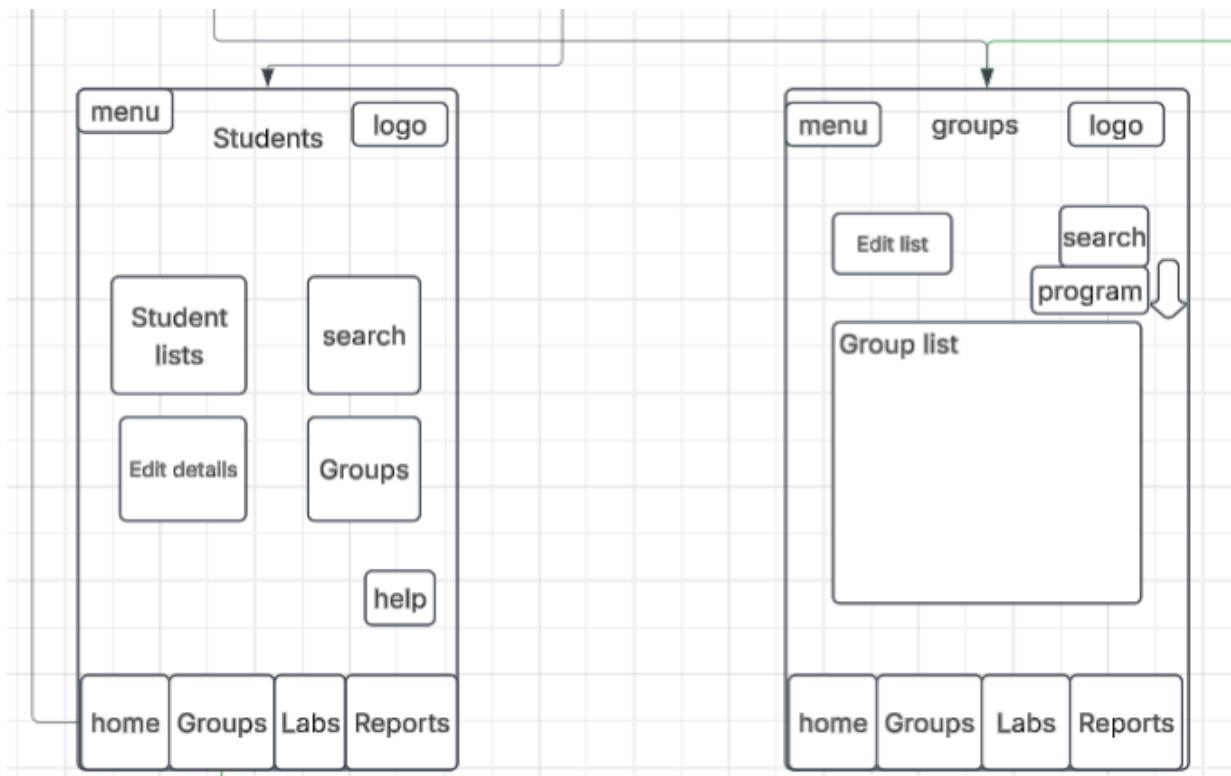
STAFF (LECTURER) LOGIN SCREEN





STAFF FRONT SCREEN

STAFF NAVIGATION



3.2 ViewModel + LiveData

Holds the state currently shown on screen and survives configuration changes such as rotation. The View observes this layer's LiveData and updates automatically when it changes.

3.3 Repository

The single source of truth for the app's data. It is the only class that knows about both Room and Retrofit, so the ViewModel does not need to know whether data came from the phone or the network. All repository work runs off the main thread to keep the UI responsive.

3.4 Room and Retrofit

Room is the local SQLite database — it stores confirmed student records and the queue of offline pending operations. Retrofit is the HTTP client that communicates with the backend API. The Repository decides which of the two to use for a given request.

3.5 WorkManager

Shown as the label between Room and the API, WorkManager runs in the background to replay queued offline operations against the server once connectivity returns, applying retry and backoff so that offline changes are not lost.

3.6 Node.js / Express API

The backend layer that authenticates every request, checks the caller's role (student or lecturer) and ownership of the record being accessed, and validates input before it reaches the database. This is the security boundary of the system — relying on the Android UI alone to hide restricted buttons is not sufficient.

3.7 MySQL / InnoDB

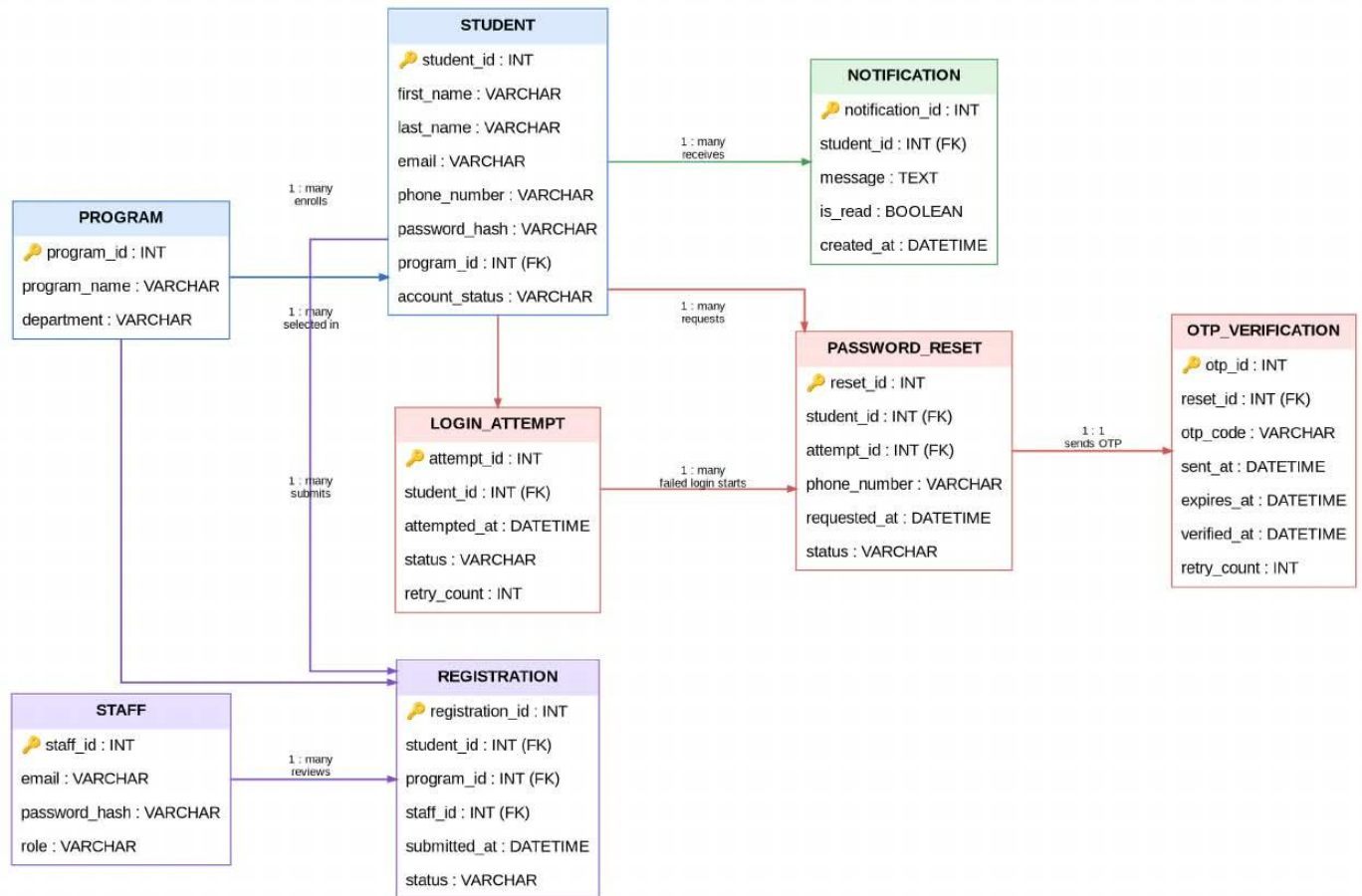
The permanent relational data store for students, accounts, groups and the pending-operations audit trail. InnoDB's transaction support is what allows the 15-member group capacity limit to be enforced safely even under simultaneous requests.

3.8 Governing rule

The note at the bottom of the diagram states the architecture's key constraint: Android never queries MySQL directly. Every read and write passes through the Express API, so authentication, authorisation and validation are enforced consistently in one place, not duplicated or bypassed on the client.

4. Entity-Relationship Diagram (ERD)

The ERD below defines the data model that the Repository, Room and the Express API all work against. It reflects the assignment's requirements around identity, capacity limits, soft deletion and offline synchronisation.



4.1 PROGRAMMES

A small reference table listing the valid programmes a student can be enrolled in (CS, IT, DS). Each STUDENTS record links to exactly one programme via a foreign key.

4.2 GROUPS

Represents the four lab groups (G01–G04). The capacity and active_count fields support enforcing the assignment's rule that no group may exceed 15 active students, including under simultaneous requests.

4.3 STUDENTS

The central entity. student_id is an immutable primary key, kept deliberately separate from student_number so that a student's identity does not depend on a value a lecturer might later correct. student_number is unique and stored as a fixed 9-character string to preserve leading zeroes. version supports conflict detection during sync. deleted and deleted_at implement soft deletion, so a student's number stays reserved and cannot be accidentally reused. claim_code lives here (not on ACCOUNTS) because a lecturer can create a student profile before that student has registered an account, and the claim code is what lets the student later verify and claim ownership of that existing profile.

4.4 ACCOUNTS

Represents login credentials for both students and lecturers, distinguished by role. An account optionally links to a student record once that student has verified ownership via their claim code. The disabled flag allows an account to be deactivated as part of the soft-delete process, without deleting the account outright.

4.5 PENDING_OPS

The offline operations queue described in the architecture diagram's WorkManager step. Each row records a single change (create, edit, transfer or delete) with a unique op_id used as an idempotency key, so that retrying a sync operation after a lost response cannot apply the same change twice. base_version is compared against the current STUDENTS.version to detect conflicting edits.

4.6 Relationships summary

- **PROGRAMMES / GROUPS → STUDENTS:** Each student has one programme and (optionally) one group.
- **STUDENTS → ACCOUNTS:** Each student is owned by at most one account, and an account may exist without a linked student until it is claimed.
- **STUDENTS / ACCOUNTS → PENDING_OPS:** Every offline change is recorded as a pending operation before it is synced, linked to both the student it affects and the account that initiated it.

5. Conclusion

The MVVM pattern keeps the app's UI, state and data cleanly separated, while the layered architecture diagram shows how that separation extends all the way to a secure, validated backend and a relational database. The ERD then defines the concrete data model — identity, capacity limits, soft deletion and offline sync — that makes the assignment's core competencies (role-based CRUD, duplicate/overflow prevention, and safe offline synchronisation) achievable in practice.